

Informe Actividad 5

- **Asignatura:** Infraestructura para C. de Datos
- **Profesor:** Michael Miranda Sandoval
- **Carrera:** Ingeniería Civil en Ciencia de Datos, UTEM
- **Alumno:** Fabrizio Larco Jorquera
- **Fecha de entrega:** 27 Mayo 2026

Registro Mínimo Obligatorio

Para garantizar la reproducibilidad de los experimentos documentados en este informe, se declara el siguiente entorno de ejecución local:

- **Sistema Operativo:** Linux Mint
- **CPU:** AMD Ryzen 5 5500 (6 núcleos físicos, 12 hilos).
- **GPU:** MSI NVIDIA GeForce RTX 3050 (6GB VRAM).
- **Memoria RAM:** 32 GB
- **Versiones de Software:**
 - Python 3.10 / 3.12 (Entornos virtuales nativos).
 - Docker Engine (Nativo) / Docker Compose V2.
 - QEMU y NASM.
- **Uso de IA:** Se declara el uso de asistencia de IA exclusivamente para la depuración de dependencias de entorno en Linux, validación de sintaxis de comandos y estructuración de formato del documento.

Resumen Ejecutivo

Se logró bootear un micro sistema operativo mediante QEMU, medir empíricamente el *overhead* en cargas de procesamiento paralelo local, y diseñar arquitecturas de datos basadas en contenedores. Se construyó un pipeline reproducible con Docker utilizando la Arquitectura Medallón y una infraestructura de microservicios con persistencia mediante Docker Compose. Finalmente, se presenta el Anteproyecto Integrador enfocado en aceleración por hardware (CPU vs. GPU) para procesamiento OLAP de series temporales.

Ejercicio 1: Mini sistema booteable en ensamblador y QEMU

Objetivo y Fundamento Técnico:

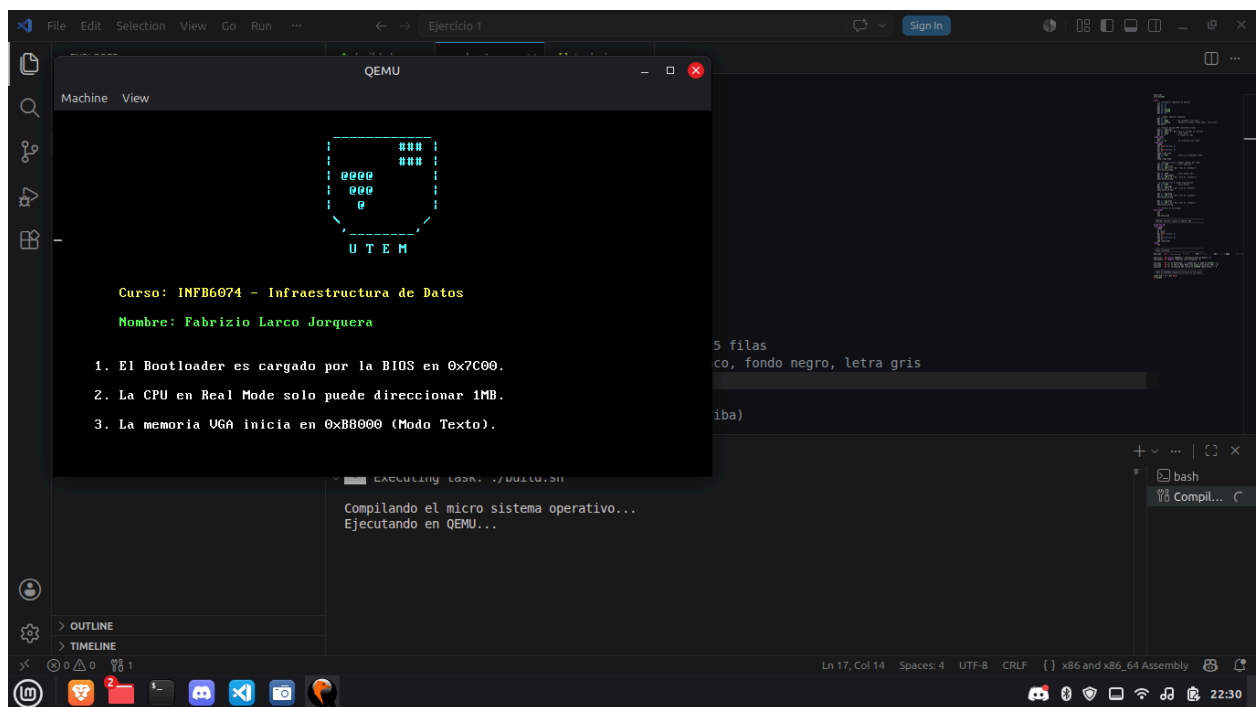
Implementar un sector de arranque funcional en modo real, eludiendo el sistema operativo. El script manipula directamente el segmento de memoria de video (VGA) para representar caracteres y atributos de color, evidenciando la abstracción inicial de hardware. Se diseñó una rutina que dibuja el emblema institucional y despliega información técnica directamente sobre la dirección física 0xB8000.

Ejecución y Comandos:

Se compiló el binario asegurando la firma de arranque de 512 bytes (0x55AA) y se emuló mediante QEMU:

```
nasm -f bin boot.asm -o boot.bin  
qemu-system-i386 -drive format=raw,file=boot.bin
```

Resultados y Análisis:



-
- **Conclusión:** Este programa permite observar de manera empírica la jerarquía de memoria física al escribir atributos de color y caracteres ASCII directamente en la memoria de la tarjeta gráfica sin intervención de un sistema operativo, demostrando control absoluto sobre el hardware en la etapa de pre-arranque.

Ejercicio 3: Paralelismo local y medición de speedup

Objetivo:

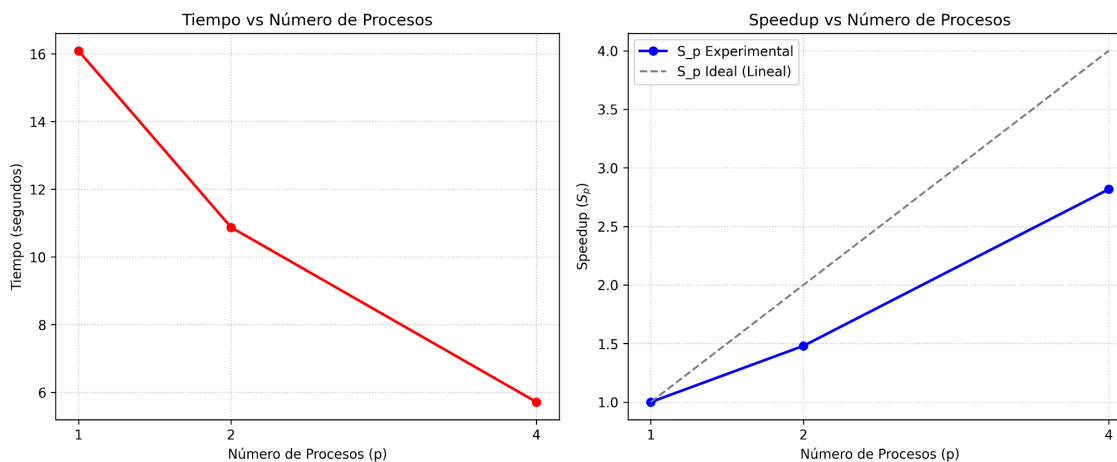
Comparar la eficiencia de un procesamiento secuencial versus uno paralelo distribuyendo la carga a través de los núcleos del procesador local, calculando el Speedup y observando las limitaciones de la escalabilidad (Overhead).

Diseño Experimental y Resultados:

- **Dataset:** 600.000 registros sintéticos.
- **Librería utilizada:** multiprocessing de Python.

Configuración	Tiempo (s)	Speedup (Sp)	Eficiencia (Ep)
Secuencial (1 proceso)	16.09	1.00x	100.00%
Paralelo (2 procesos)	10.87	1.48x	74.00%
Paralelo (4 procesos)	5.71	2.81x	70.43%

Evidencia Gráfica:



El Speedup obtenido demuestra un comportamiento no lineal. Al distribuir la carga en 4 procesos, el tiempo se reduce de 16.09s a 5.71s, logrando un Speedup de 2.81x (y no el teórico ideal de 4.0x). Esto se explica por el *overhead* asociado a la serialización de datos y al costo de conmutación de contexto por parte del sistema operativo. Las tareas de particionamiento de memoria consumen ciclos de reloj adicionales, lo que justifica por qué la eficiencia decae al 70.43% al utilizar una mayor cantidad de núcleos físicos.

Ejercicio 4: Pipeline Big Data local con Python, Docker y Parquet

Objetivo:

Implementar un pipeline administrado, aislado de las dependencias locales del host, utilizando la Arquitectura Medallón para aplicar validaciones de calidad y almacenamiento columnar.

Diseño del Pipeline (Telemetría de Servidores):

- **Zona Bronze:** Ingesta de datos crudos (CSV y JSONL) con anomalías intencionales.
- **Zona Silver:** Aplicación de 6 reglas de calidad estrictas (eliminación de duplicados, validación de rangos, integridad referencial, remoción de nulos, etc.).
- **Zona Gold:** Consultas analíticas mediante DuckDB sobre archivos Parquet particionados.

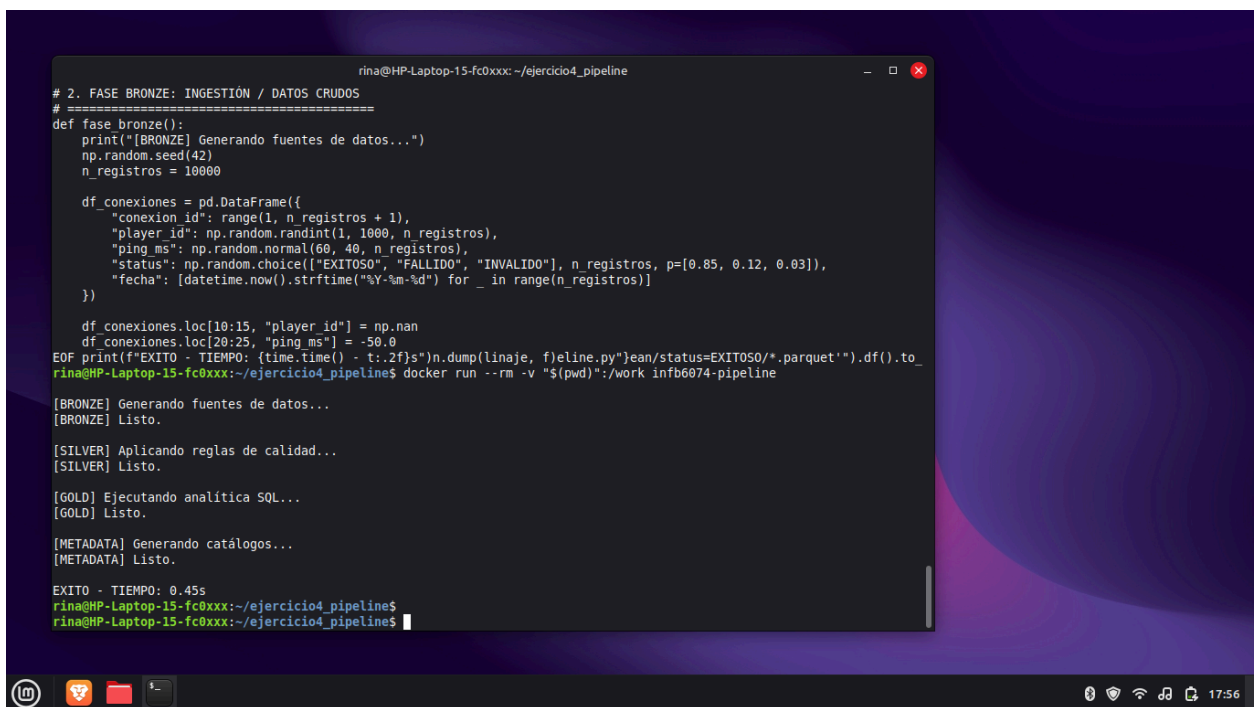
Reproducibilidad y Ejecución:

Se generó una imagen inmutable con Python 3.10 y se ejecutó montando un volumen:

```
docker build -t infb6074-pipeline .  
docker run --rm -v "$(pwd)":/work infb6074-pipeline
```

Análisis Arquitectónico:

Docker transformó un script de Python en una infraestructura analítica portátil. El mapeo de volúmenes permitió que el contenedor escribiera el catálogo de datos y los archivos Parquet directamente en el disco físico sin instalar dependencias invasivas en el sistema base, garantizando un ambiente 100% reproducible.



```
rina@HP-Laptop-15-fc0xxx: ~/ejercicio4_pipeline  
# 2. FASE BRONZE: INGESTIÓN / DATOS CRUDOS  
# =====  
def fase_bronze():  
    print("[BRONZE] Generando fuentes de datos...")  
    np.random.seed(42)  
    n_registros = 10000  
  
    df_conexiones = pd.DataFrame({  
        "conexion_id": range(1, n_registros + 1),  
        "player_id": np.random.randint(1, 1000, n_registros),  
        "ping_ms": np.random.normal(60, 40, n_registros),  
        "status": np.random.choice(["EXITO50", "FALLIDO", "INVALIDO"], n_registros, p=[0.85, 0.12, 0.03]),  
        "fecha": [datetime.now().strftime("%Y-%m-%d") for _ in range(n_registros)]  
    })  
  
    df_conexiones.loc[10:15, "player_id"] = np.nan  
    df_conexiones.loc[20:25, "ping_ms"] = -50.0  
EOF print(f"EXITO - TIEMPO: {time.time() - t:.2f}s")n.dump(linaje, f)eline.py}ean/status=EXITO50/*.parquet').df().to_  
rina@HP-Laptop-15-fc0xxx:~/ejercicio4_pipeline$ docker run --rm -v "$(pwd)":/work infb6074-pipeline  
[BRONZE] Generando fuentes de datos...  
[BRONZE] Listo.  
  
[SILVER] Aplicando reglas de calidad...  
[SILVER] Listo.  
  
[GOLD] Ejecutando analítica SQL...  
[GOLD] Listo.  
  
[METADATA] Generando catálogos...  
[METADATA] Listo.  
  
EXITO - TIEMPO: 0.45s  
rina@HP-Laptop-15-fc0xxx:~/ejercicio4_pipeline$  
rina@HP-Laptop-15-fc0xxx:~/ejercicio4_pipeline$
```

Ejercicio 5: Mini infraestructura con Docker Compose

Objetivo:

Orquestar una arquitectura de microservicios simulando un entorno donde el cómputo y el almacenamiento operan como entidades separadas pero comunicadas por una red privada segura.

Topología y Comandos:

Mediante un archivo docker-compose.yml, se desplegaron:

1. Servidor Relacional (PostgreSQL 15).
2. Contenedor de Cómputo Analítico (Python 3.10).
3. Red Bridge privada (data_net).
4. Volumen persistente (pgdata).

El despliegue interactivo se realizó con:

```
docker compose up --build
```

Resultados y Discusión Técnica:

El motor analítico logró autenticarse a través de la red de contenedores, poblar la base de datos y extraer información agregada. Docker Compose resuelve eficientemente la complejidad de las redes internas y la inyección de variables de entorno. Adicionalmente, el volumen asignado garantiza la retención de datos. Sin embargo, esta arquitectura local carece de escalabilidad elástica y tolerancia a fallos, lo que en producción exigiría orquestadores de grado empresarial como Kubernetes.

```
rina@HP-Laptop-15-fcbxxx: ~/ejercicio5_compose
db-1 | 2026-05-26 22:32:10.283 UTC [48] LOG:  received fast shutdown request
db-1 | 2026-05-26 22:32:10.286 UTC [48] LOG:  aborting any active transactions
db-1 | 2026-05-26 22:32:10.289 UTC [48] LOG:  background worker "logical replication launcher" (PID 54) exited with exit code 1
db-1 | 2026-05-26 22:32:10.289 UTC [49] LOG:  shutting down
db-1 | 2026-05-26 22:32:10.292 UTC [49] LOG:  checkpoint starting: shutdown immediate
db-1 | 2026-05-26 22:32:10.349 UTC [49] LOG:  checkpoint complete: wrote 922 buffers (5.6%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.016 s, sync=0.034 s, to
tal=0.060 s; sync files=301, longest=0.003 s, average=0.001 s; distance=4239 kB, estimate=4239 kB
db-1 | 2026-05-26 22:32:10.357 UTC [48] LOG:  database system is shut down
db-1 | done
db-1 | server stopped
db-1 |
db-1 | PostgreSQL init process complete; ready for start up.
db-1 |
db-1 | 2026-05-26 22:32:10.408 UTC [1] LOG:  starting PostgreSQL 15.18 (Debian 15.18-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-b
it
db-1 | 2026-05-26 22:32:10.408 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-05-26 22:32:10.408 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
db-1 | 2026-05-26 22:32:10.412 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-05-26 22:32:10.417 UTC [64] LOG:  database system was shut down at 2026-05-26 22:32:10 UTC
db-1 | 2026-05-26 22:32:10.424 UTC [1] LOG:  database system is ready to accept connections
python_app-1 | Esperando a que la base de datos inicie...
python_app-1 | Conexión exitosa a PostgreSQL a través de la red interna!
python_app-1 | Generando dataset...
python_app-1 | Cargando datos a PostgreSQL...
python_app-1 | Consultando la base de datos...
python_app-1 |
python_app-1 | --- TOP 5 JUGADORES ---
python_app-1 | jugador_id  puntaje
python_app-1 | 0           76      4987
python_app-1 | 1           95      4975
python_app-1 | 2           78      4959
python_app-1 | 3           18      4943
python_app-1 | 4           63      4898
python_app-1 | -----
python_app-1 |
python_app-1 | Resultados guardados en resultado_consulta_compose.csv
python_app-1 | python_app-1 exited with code 0

Enable Watch
```

Ejercicio 6: Evaluación comparativa de arquitecturas

Criterio (Peso)	Arq. A (Monolítico local)	Arq. B (Pipeline Docker)	Arq. C (Compose + BD)
Reproducibilidad (20%)	1	5	5
Escalabilidad (15%)	1	2	4
Gobierno de Datos (15%)	1	4	4
Comp. Operativa (15%)	5	3	2
Aislamiento (10%)	1	4	5
Costo Computacional (10%)	5	4	3
Madurez Equipo (5%)	5	3	2
Mantenibilidad (10%)	2	4	4
PUNTAJE FINAL	2.20	3.70	3.80

Para analítica *batch* local orientada a un equipo de Data Science, la **Arquitectura B** es óptima por su excelente balance entre reproducibilidad y bajo consumo de recursos de RAM. Para sistemas con crecimiento de volumen continuo, la **Arquitectura C** es el paso transicional correcto, al separar estructuralmente el cómputo del almacenamiento, preparando una futura migración hacia la nube.

PARTE II: ANTEPROYECTO INTEGRADOR

Evolución Histórica del Rendimiento de Héroes en Overwatch mediante GPU.

Planteamiento del Problema

En el análisis competitivo de los E-Sports, comprender la evolución del "meta" requiere evaluar el rendimiento de los personajes a lo largo del tiempo. Analizar la variación de estadísticas (como el *win rate*, selecciones competitivas o el impacto en sanación/daño por temporada) desde lanzamientos tempranos, como la introducción de Junkrat en la beta de 2015, hasta la actualidad, exige agrupar y cruzar decenas de millones de registros de telemetría. Utilizar motores tradicionales de procesamiento de datos en Python (como Pandas) obliga a ejecutar estos cálculos históricos de forma secuencial sobre el procesador central (CPU). Esto genera un cuello de botella computacional severo, haciendo que el análisis de métricas temporales masivas sea ineficiente y demore horas de cómputo en hardware local estándar.

Arquitectura Propuesta

Se propone una arquitectura de procesamiento analítico (OLAP) acelerada por hardware, delegando el cálculo matricial y las agrupaciones históricas a la tarjeta gráfica (GPU). Se utilizará la suite **NVIDIA RAPIDS (cuDF)**, que permite mantener la sintaxis estándar de manipulación de DataFrames de Python, pero ejecuta las operaciones directamente sobre los miles de núcleos CUDA de la GPU local. Esto logra un paralelismo masivo capaz de resolver líneas de tiempo complejas en fracciones de segundo, emulando el rendimiento de un clúster distribuido en una sola máquina.

4. Generación de Datos Sintéticos (Prueba de Concepto)

Debido a la ausencia de APIs públicas masivas y accesibles por parte de la desarrolladora (Blizzard Entertainment), se construyó un pipeline generador de Datos Sintéticos como Prueba de Concepto.

Para asegurar la validez técnica del experimento analítico, las distribuciones estadísticas del generador (Pick Rate, Win Rate, Damage/Healing output) fueron calibradas utilizando las métricas agregadas oficiales publicadas en el portal *Battle.net Hero Statistics*. El script implementado mediante numpy es capaz de generar 5 millones de registros de partidas hiperrealistas en menos de 10 segundos, proveyendo el volumen de datos crudos necesario para estresar la jerarquía de memoria del hardware.

5. Metodología de Experimentación

Se evaluará un dataset de telemetría escalable (de 1 a 20 millones de filas) en tres escenarios computacionales:

1. **Escenario Base (Secuencial):** Procesamiento histórico con pandas en 1 hilo del procesador AMD Ryzen 5 5500.
2. **Escenario Optimizado (Multihilo):** Procesamiento con librerías de vanguardia (DuckDB o Polars) utilizando los 6 núcleos físicos y 12 hilos de la CPU.
3. **Escenario Acelerado (Paralelismo Masivo):** Procesamiento mediante cuDF explotando los núcleos CUDA de la gráfica MSI RTX 3050.

6. Análisis de Riesgos y Limitantes (Cuello de Botella)

El riesgo crítico inherente a la arquitectura propuesta es el límite estricto de la memoria de video (VRAM). Mientras que la CPU puede mitigar la saturación de memoria apoyándose en el archivo de paginación del disco duro (Swap), los 6GB de VRAM de la tarjeta gráfica representan una barrera inflexible. Si el volumen del DataFrame histórico descomprimido supera este límite, la ejecución colapsará (*Out of Memory*). El proyecto documentará empíricamente el umbral exacto de datos históricos donde la aceleración de la GPU justifica este riesgo operativo frente a la resiliencia secuencial de la CPU.